Assignment No. 2 — Practical submission

Code Snippets and Explanation

Plant Water Advisor
com.plantwise.wateradvisor

Student: Ahmed Asif
Date: 29 October 2026
Roll number: _____ Course: _____

1. Model loading and inference — MainActivity.java

This is the whole prediction path. The class comment states the formula. `loadModelFile` maps the asset. `predictWater` builds the input and output arrays and calls `Interpreter.run`. `onDestroy` closes the native interpreter.

```
package com.plantwise.wateradvisor;

import android.content.res.AssetFileDescriptor;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import org.tensorflow.lite.Interpreter;

import java.io.FileInputStream;
import java.io.IOException;
import java.nio.MappedByteBuffer;
import java.nio.channels.FileChannel;

/**
 * Plant Water Advisor.
 *
 * The user types a soil-dryness score from 0 to 10. A TensorFlow Lite model
 * in assets/water.tflite runs on the phone and returns how many cups of water
 * to give the plant. The model is one dense layer:
 *
 *     cups = (2.0 * dryness) + 0.0
 *
 * Nothing is sent to a server. The same load / input / run / output pattern
 * is used by the reference project NumberGuessingAppTflite.
 */
public class MainActivity extends AppCompatActivity {

    /** File name inside app/src/main/assets. */
    private static final String MODEL_FILE = "water.tflite";

    private EditText drynessInput;
    private TextView resultView;

    /**
     * The TFLite Interpreter holds the loaded model and runs one prediction
     * at a time. It is created in onCreate and closed in onDestroy.
     */
    private Interpreter interpreter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

        drynessInput = findViewById(R.id.drynessInput);
        resultView = findViewById(R.id.resultView);
        Button predictButton = findViewById(R.id.predictButton);

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (view, insets) -> {
            Insets bars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            view.setPadding(bars.left + 24, bars.top + 24, bars.right + 24, bars.bottom + 24);
            return insets;
        });

        try {
```

```
            // Map the .tflite file into memory, then build the interpreter.
            interpreter = new Interpreter(loadModelFile());
        } catch (IOException exception) {
            resultView.setText(R.string.model_missing);
            predictButton.setEnabled(false);
            return;
        }

        predictButton.setOnClickListener(view -> predictWater());
    }

    /**
     * Reads the number, checks it, runs the model, and shows the cups.
     */
    private void predictWater() {
        String raw = drynessInput.getText().toString().trim();
        if (raw.isEmpty()) {
            resultView.setText(R.string.error_empty);
            return;
        }

        float dryness;
        try {
            dryness = Float.parseFloat(raw);
        } catch (NumberFormatException exception) {
            resultView.setText(R.string.error_number);
            return;
        }

        if (dryness < 0f || dryness > 10f) {
            resultView.setText(R.string.error_range);
            return;
        }

        // Input tensor shape [1]: one float feature (dryness).
        float[] input = new float[]{dryness};

        // Output tensor shape [1][1]: one float prediction (cups of water).
        float[][] output = new float[1][1];

        // Copies input into the model, runs the dense layer, writes output.
        interpreter.run(input, output);

        float cups = output[0][0];
        resultView.setText(getString(R.string.result_cups, cups));
    }

    /**
     * Opens assets/water.tflite and returns a read-only mapped buffer.
     * AssetFileDescriptor is required because Interpreter needs a MappedByteBuffer,
     * not a normal InputStream. The asset must be stored uncompressed.
     */
    private MappedByteBuffer loadModelFile() throws IOException {
        AssetFileDescriptor descriptor = getAssets().openFd(MODEL_FILE);
        FileInputStream inputStream = new FileInputStream(descriptor.getFileDescriptor());
        FileChannel channel = inputStream.getChannel();
        long start = descriptor.getStartOffset();
        long length = descriptor.getDeclaredLength();
        return channel.map(FileChannel.MapMode.READ_ONLY, start, length);
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        if (interpreter != null) {
```

```
            interpreter.close();
            interpreter = null;
        }
    }
}
```

2. What each ML step does

- `MODEL_FILE` is the asset name. The real path is `app/src/main/assets/water.tflite`.
- `Interpreter` is created once, after the layout exists, and is reused for every tap.
- `float[] input` has length 1 because the dense layer expects one feature.
- `float[][] output` is `[1][1]` because the Java API includes the batch dimension.
- `output[0][0]` is the only predicted number. It is formatted with one decimal place.
- Invalid text returns before `run`, so a bad string cannot crash `parseFloat` the way the reference app can.

3. User interface — activity_main.xml

A vertical `LinearLayout` holds the question, the numeric field, the button, and the result. The result view is the only place the model output appears.

```
<?xml version="1.0" encoding="utf-8"?>
<!-- One screen: dryness input, predict button, and the model result. -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:text="@string/prompt"
        android:textColor="@color/text_dark"
        android:textSize="22sp"
        android:textStyle="bold" />

    <!-- Raw dryness typed by the user. It is parsed to float before inference. -->
    <EditText
        android:id="@+id/drynessInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:hint="@string/hint_dryness"
        android:importantForAutofill="no"
        android:inputType="numberDecimal"
        android:textSize="20sp" />

    <Button
        android:id="@+id/predictButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="@string/predict"
        android:textSize="18sp" />

    <!-- Filled after Interpreter.run returns. -->
    <TextView
        android:id="@+id/resultView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:text="@string/result_hint"
        android:textColor="@color/text_dark"
        android:textSize="20sp" />

</LinearLayout>
```

4. Dependency and uncompressed model — app/build.gradle

`tensorflow-lite:2.16.1` supplies the `Interpreter` class. `noCompress 'tflite'` keeps the asset stored so `openFd` can memory-map it. A compressed asset would fail at load time.

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace 'com.plantwise.wateradvisor'
    compileSdk 35

    defaultConfig {
        applicationId "com.plantwise.wateradvisor"
        minSdk 24
        targetSdk 35
        versionCode 1
        versionName "1.0"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-
rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_11
        targetCompatibility JavaVersion.VERSION_11
    }
    // A compressed .tflite asset cannot be memory-mapped. Keep it stored as-is.
    androidResources {
        noCompress 'tflite'
    }
}

dependencies {
    // Interpreter API used to load water.tflite and run on-device inference.
    implementation 'org.tensorflow:tensorflow-lite:2.16.1'
    implementation libs.androidx.core.ktx
    implementation libs.androidx.appcompat
    implementation libs.material
}
```

5. Reference versus this project

	NumberGuessingAppTflite	Plant Water Advisor
Model file	`linear.tflite`	`water.tflite`
Function	about `2x − 1`	`2x`, reported as cups
Input check	none	required, numeric, 0 to 10
Result text	raw float	“Recommended water: N cups”
Interpreter lifetime	left open	closed in `onDestroy`

Both apps use the same integration idea: an uncompressed `.tflite` asset, a mapped buffer, one input float, one output float, and a text view.